Petroleum Pipeline

dbt + Snowflake Data Pipeline

Bronze - Silver - Gold Medallion Architecture

Star Schema (Facts & Dimensions)

Incremental Loading | SCD Type 2 Snapshots

CI/CD with GitHub Actions | Airflow Orchestration

Author: Rahul Tiwary

Repository: github.com/tiwaryrahul/Petroleum-

Date: March 2026

Table of Contents

1. Project Overview & Architecture
2. Snowflake Configuration
3. Project Structure
4. Bronze Layer - Raw Data Ingestion
5. Silver Layer - Cleaning & Enrichment
6. Gold Layer - Star Schema
7. Snapshots (SCD Type 2)
8. Tests & Data Quality
9. Seeds - Sample Data
10. Macros
11. CI/CD - GitHub Actions
12. Orchestration - Apache Airflow
13. End-to-End Workflow
14. Commands Reference

1. Project Overview & Architecture

This project implements a complete data pipeline for petroleum industry financial audit and operational data using dbt (Data Build Tool) on Snowflake. The architecture follows the Bronze-Silver-Gold medallion pattern with a star schema in the Gold layer.

Architecture Flow

```
S3 (petroleumdb) + Snowflake (PETROLEUM_AUDIT.BRONZE)
      |
   BRONZE LAYER (Incremental loads, raw data ingestion)
      |
   SILVER LAYER (Cleaning, Standardization, Business Logic)
      |
   GOLD LAYER (Star Schema - Facts & Dimensions)
```

Key Features

- Medallion Architecture: Bronze -> Silver -> Gold
- Incremental Loading: Watermark & key-based strategies
- Star Schema: 5 Fact tables + 6 Dimension tables
- SCD Type 2: Vendor snapshot with check strategy
- Data Quality: 28 schema tests + 4 custom SQL tests
- CI/CD: GitHub Actions (daily scheduled + manual trigger)
- Orchestration: Apache Airflow DAG
- Custom Schema Macro: Clean schema names (bronze/silver/gold)

2. Snowflake Configuration

Connection Details

Setting	Value
Account	KKTZVDG-MO79591
User	RAHULTIWARY10
Role	ACCOUNTADMIN
Warehouse	COMPUTE_WH
Database	PETROLEUM_DB
Source DB	PETROLEUM_AUDIT
Auth	Password (env var)

Schemas in PETROLEUM_DB

Schema	Purpose
BRONZE	Raw/ingested data models
SILVER	Cleaned, enriched data
GOLD	Star schema (facts & dims)
SNAPSHOTS	SCD Type 2 snapshots
PUBLIC	Seeds and default schema

profiles.yml

```
petroleum_pipeline:
  target: dev
  outputs:
    dev:
      type: snowflake
      account: KKTZVDG-MO79591
      user: RAHULTIWARY10
      password: "{{ env_var('SNOWFLAKE_PASSWORD') }}"
      role: ACCOUNTADMIN
      warehouse: COMPUTE_WH
      database: PETROLEUM_DB
      schema: public
      threads: 4
```

dbt_project.yml

```
# Name your project! Project names should contain only lowercase characters
# and underscores. A good package name should reflect your organization's
# name or the intended use of these models
name: 'petroleum_pipeline'
version: '1.0.0'

# This setting configures which "profile" dbt uses for this project.
profile: 'petroleum_pipeline'

# These configurations specify where dbt should look for different types of files.
```

```
# The `model-paths` config, for example, states that models in this project can be
# found in the "models/" directory. You probably won't need to change these!
model-paths: ["models"]
analysis-paths: ["analyses"]
test-paths: ["tests"]
seed-paths: ["seeds"]
macro-paths: ["macros"]
snapshot-paths: ["snapshots"]

clean-targets:         # directories to be removed by `dbt clean`
  - "target"
  - "dbt_packages"


# Configuring models
# Full documentation: https://docs.getdbt.com/docs/configuring-models

# In this example config, we tell dbt to build all models in the example/
# directory as views. These settings can be overridden in the individual model
# files using the `{{ config(...) }}` macro.
models:
  petroleum_pipeline:
    bronze:
      +materialized: view
      +schema: bronze
    silver:
      +materialized: view
      +schema: silver
    gold:
      +materialized: table
      +schema: gold
```

3. Project Structure

```
petroleum_pipeline/
|-- dbt_project.yml
|-- profiles.yml
|-- models/
|   |-- bronze/
|   |   |-- _bronze_schema.yml
|   |   |-- _bronze_sources.yml
|   |   |-- bronze_gl.sql
|   |   |-- bronze_vendor_master.sql
|   |   |-- bronze_customer_master.sql
|   |   |-- bronze_product_master.sql
|   |   |-- bronze_subsidiary_master.sql
|   |   |-- bronze_chart_of_accounts.sql
|   |   |-- bronze_general_ledger_transactions.sql
|   |   |-- bronze_well_production.sql
|   |   |-- bronze_pipeline_operations.sql
|   |-- silver/
|   |   |-- _silver_schema.yml
|   |   |-- gl_transactions_enriched.sql
|   |   |-- vendor_payments.sql
|   |   |-- silver_general_ledger.sql
|   |   |-- silver_vendor_master.sql
|   |   |-- silver_customer_master.sql
|   |   |-- silver_product_master.sql
|   |   |-- silver_subsidiary_master.sql
|   |   |-- silver_chart_of_accounts.sql
|   |   |-- silver_well_production.sql
|   |   |-- silver_pipeline_operations.sql
|   |-- gold/
|       |-- _gold_schema.yml
|       |-- fact_general_ledger.sql
|       |-- fact_vendor_payments.sql
|       |-- fact_revenue.sql
|       |-- gold_daily_production_summary.sql
|       |-- gold_pipeline_health.sql
|       |-- dim_vendor.sql
|       |-- dim_customer.sql
|       |-- dim_product.sql
|       |-- dim_subsidiary.sql
|       |-- dim_chart_of_accounts.sql
|       |-- dim_date.sql
|-- snapshots/
|   |-- vendor_snapshot.sql
|-- seeds/
|   |-- seed_well_production.csv
|   |-- seed_pipeline_operations.csv
|   |-- seed_properties.yml
|-- tests/
|   |-- trial_balance.sql
|   |-- assert_no_negative_oil_volume.sql
|   |-- assert_no_negative_pressure_drop.sql
|   |-- assert_water_cut_in_range.sql
|-- macros/
|   |-- generate_schema_name.sql
|-- analyses/
```

4. Bronze Layer - Raw Data Ingestion

The Bronze layer ingests raw data from PETROLEUM_AUDIT.BRONZE using incremental models. Each model uses a unique key to prevent duplicates. The GL model uses watermark-based incremental loading on posting_date.

Source Definition

_bronze_sources.yml

```
version: 2

sources:
  - name: bronze
    database: PETROLEUM_AUDIT
    schema: BRONZE

    tables:
      - name: general_ledger_transactions
      - name: vendor_master
      - name: customer_master
      - name: product_master
      - name: subsidiary_master
      - name: chart_of_accounts
```

Schema & Tests

_bronze_schema.yml

```
version: 2

models:
  - name: bronze_well_production
    description: Raw well production data passthrough
    columns:
      - name: well_id
        description: Unique well identifier
        tests:
          - not_null
      - name: production_date
        tests:
          - not_null
      - name: oil_volume_bbl
        tests:
          - not_null
      - name: status
        tests:
          - accepted_values:
              values: ['ACTIVE', 'SHUT_IN', 'ABANDONED', 'SUSPENDED']

  - name: bronze_pipeline_operations
    description: Raw pipeline sensor and operational data
    columns:
      - name: pipeline_id
        tests:
          - not_null
      - name: segment_id
        tests:
          - not_null
      - name: reading_timestamp
        tests:
```

```
          - not_null
      - name: status
        tests:
          - accepted_values:
              values: ['OPERATIONAL', 'MAINTENANCE', 'SHUTDOWN', 'ALARM']
```

bronze_gl.sql - Watermark incremental on posting_date

bronze_gl.sql

```
{{ config(
    materialized='incremental',
    unique_key='journal_id'
) }}

SELECT *
FROM {{ source('bronze','general_ledger_transactions') }}

{% if is_incremental() %}
WHERE posting_date >
(
    SELECT MAX(posting_date)
    FROM {{ this }}
)
{% endif %}
```

bronze_vendor_master.sql - Incremental by vendor_id

bronze_vendor_master.sql

```
{{ config(
    materialized='incremental',
    unique_key='vendor_id'
) }}

SELECT *
FROM {{ source('bronze', 'vendor_master') }}

{% if is_incremental() %}
WHERE vendor_id NOT IN (
    SELECT vendor_id FROM {{ this }}
)
{% endif %}
```

bronze_customer_master.sql - Incremental by customer_id

bronze_customer_master.sql

```
{{ config(
    materialized='incremental',
    unique_key='customer_id'
) }}

SELECT *
FROM {{ source('bronze', 'customer_master') }}

{% if is_incremental() %}
WHERE customer_id NOT IN (
    SELECT customer_id FROM {{ this }}
)
{% endif %}
```

bronze_product_master.sql - Incremental by product_id

bronze_product_master.sql

```
{{ config(
    materialized='incremental',
    unique_key='product_id'
) }}

SELECT *
FROM {{ source('bronze', 'product_master') }}

{% if is_incremental() %}
WHERE product_id NOT IN (
    SELECT product_id FROM {{ this }}
)
{% endif %}
```

bronze_subsidiary_master.sql - Incremental by subsidiary_id

bronze_subsidiary_master.sql

```
{{ config(
    materialized='incremental',
    unique_key='subsidiary_id'
) }}

SELECT *
FROM {{ source('bronze', 'subsidiary_master') }}

{% if is_incremental() %}
WHERE subsidiary_id NOT IN (
    SELECT subsidiary_id FROM {{ this }}
)
{% endif %}
```

bronze_chart_of_accounts.sql - Incremental by account_number

bronze_chart_of_accounts.sql

```
{{ config(
    materialized='incremental',
    unique_key='account_number'
) }}

SELECT *
FROM {{ source('bronze', 'chart_of_accounts') }}

{% if is_incremental() %}
WHERE account_number NOT IN (
    SELECT account_number FROM {{ this }}
)
{% endif %}
```

bronze_general_ledger_transactions.sql - View passthrough

bronze_general_ledger_transactions.sql

```
{{ config(materialized='view') }}

select
    journal_id,
    line_id,
    posting_date,
    document_date,
    subsidiary_id,
    account_number,
    account_name,
    debit_amount,
    credit_amount,
    customer_id,
    vendor_id,
    product_id,
    quantity,
    created_by,
    approval_status,
    error_flag,
    error_type
from {{ source('bronze', 'general_ledger_transactions') }}
```

bronze_well_production.sql - View from seed

bronze_well_production.sql

```
{{ config(materialized='view') }}

select
    well_id,
    well_name,
    production_date,
    oil_volume_bbl,
    gas_volume_mcf,
    water_volume_bbl,
    wellhead_pressure_psi,
    wellhead_temperature_f,
    choke_size,
    status,
    _loaded_at
from {{ ref('seed_well_production') }}
```

bronze_pipeline_operations.sql - View from seed

bronze_pipeline_operations.sql

```
{{ config(materialized='view') }}

select
    pipeline_id,
    pipeline_name,
    segment_id,
    reading_timestamp,
    inlet_pressure_psi,
    outlet_pressure_psi,
    flow_rate_bpd,
    temperature_f,
    status,
    _loaded_at
from {{ ref('seed_pipeline_operations') }}
```

5. Silver Layer - Cleaning & Enrichment

The Silver layer performs data cleaning (null handling, trimming), standardization (uppercase, initcap), and business logic (calculated fields, classifications, joins).

Schema & Tests

_silver_schema.yml

```
version: 2

models:
  - name: silver_well_production
    description: Cleansed well production with calculated metrics
    columns:
      - name: well_id
        tests:
          - not_null
      - name: production_date
        tests:
          - not_null
      - name: water_cut_pct
        description: Water cut percentage
        tests:
          - not_null
      - name: total_liquid_bbl
        tests:
          - not_null

  - name: gl_transactions_enriched
    description: Enriched GL transactions joined with all master data
    columns:
      - name: journal_entry_id
        tests:
          - not_null
      - name: account_number
        tests:
          - relationships:
              to: source('bronze','chart_of_accounts')
              field: account_number

  - name: silver_pipeline_operations
    description: Cleansed pipeline data with pressure drop analysis
    columns:
      - name: pipeline_id
        tests:
          - not_null
      - name: pressure_drop_psi
        tests:
          - not_null
      - name: pressure_drop_severity
        tests:
          - accepted_values:
              values: ['NORMAL', 'MEDIUM', 'HIGH']
```

gl_transactions_enriched.sql

gl_transactions_enriched.sql

```
{{ config(materialized='table') }}

SELECT

    gl.journal_id AS journal_entry_id,
    gl.line_id,
    gl.posting_date,

    gl.account_number,
    coa.account_name,
    coa.account_type,

    gl.debit_amount,
    gl.credit_amount,

    gl.customer_id,
    cm.customer_name,

    gl.vendor_id,
    vm.vendor_name,

    gl.product_id,
    pm.product_name,

    pm.standard_cost_per_barrel,

    gl.quantity,
    gl.created_by

FROM {{ source('bronze','general_ledger_transactions') }} gl

LEFT JOIN {{ source('bronze','chart_of_accounts') }} coa
    ON gl.account_number = coa.account_number

LEFT JOIN {{ source('bronze','customer_master') }} cm
    ON gl.customer_id = cm.customer_id

LEFT JOIN {{ source('bronze','vendor_master') }} vm
    ON gl.vendor_id = vm.vendor_id

LEFT JOIN {{ source('bronze','product_master') }} pm
    ON gl.product_id = pm.product_id
```

vendor_payments.sql

vendor_payments.sql

```
{{ config(materialized='table') }}

SELECT

    journal_entry_id,
    vendor_id,
    vendor_name,

    SUM(debit_amount) AS payment_amount,

    COUNT(*) AS transactions

FROM {{ ref('gl_transactions_enriched') }}

WHERE vendor_id IS NOT NULL

GROUP BY
    journal_entry_id,
    vendor_id,
    vendor_name
```

silver_general_ledger.sql

silver_general_ledger.sql

```
{{ config(materialized='view') }}

select
    gl.journal_id,
    gl.line_id,
    gl.posting_date,
    gl.document_date,
    gl.subsidiary_id,
    sub.subsidiary_name,
    sub.functional_currency,
    gl.account_number,
    coa.account_name,
    coa.account_type,
    coa.sub_type,
    coa.normal_balance,
    coa.financial_statement_category,
    coalesce(gl.debit_amount, 0) as debit_amount,
    coalesce(gl.credit_amount, 0) as credit_amount,
    coalesce(gl.debit_amount, 0) - coalesce(gl.credit_amount, 0) as net_amount,
    gl.customer_id,
    cust.customer_name,
    cust.industry as customer_industry,
    gl.vendor_id,
    vend.vendor_name,
    vend.service_type as vendor_service_type,
    gl.product_id,
    prod.product_name,
    prod.category as product_category,
    prod.standard_cost_per_barrel,
    gl.quantity,
    case
        when gl.quantity > 0 and prod.standard_cost_per_barrel is not null
        then gl.quantity * prod.standard_cost_per_barrel
        else null
    end as standard_cost_total,
    gl.created_by,
    upper(trim(gl.approval_status)) as approval_status,
    upper(trim(gl.error_flag)) as error_flag,
    gl.error_type,
    case
        when upper(trim(gl.error_flag)) = 'Y' then true
        else false
    end as is_error,
    case
        when upper(trim(gl.approval_status)) = 'APPROVED' then true
        else false
    end as is_approved
from {{ ref('bronze_gl') }} gl
left join {{ ref('bronze_subsidiary_master') }} sub
    on gl.subsidiary_id = sub.subsidiary_id
left join {{ ref('bronze_chart_of_accounts') }} coa
    on gl.account_number = coa.account_number
left join {{ ref('bronze_customer_master') }} cust
    on gl.customer_id = cust.customer_id
left join {{ ref('bronze_vendor_master') }} vend
    on gl.vendor_id = vend.vendor_id
left join {{ ref('bronze_product_master') }} prod
    on gl.product_id = prod.product_id
```

silver_vendor_master.sql

silver_vendor_master.sql

```
{{ config(materialized='view') }}

select
    vendor_id,
    initcap(trim(vendor_name)) as vendor_name,
    upper(trim(service_type)) as service_type,
    initcap(trim(location)) as location
from {{ ref('bronze_vendor_master') }}
where vendor_id is not null
```

silver_customer_master.sql

silver_customer_master.sql

```
{{ config(materialized='view') }}

select
    customer_id,
    initcap(trim(customer_name)) as customer_name,
    coalesce(credit_limit, 0) as credit_limit,
    case
        when credit_limit >= 1000000 then 'TIER_1'
        when credit_limit >= 500000 then 'TIER_2'
        when credit_limit >= 100000 then 'TIER_3'
        else 'TIER_4'
    end as credit_tier,
    upper(trim(industry)) as industry,
    initcap(trim(location)) as location
from {{ ref('bronze_customer_master') }}
where customer_id is not null
```

silver_product_master.sql

silver_product_master.sql

```
{{ config(materialized='view') }}

select
    product_id,
    initcap(trim(product_name)) as product_name,
    upper(trim(category)) as category,
    coalesce(standard_cost_per_barrel, 0) as standard_cost_per_barrel
from {{ ref('bronze_product_master') }}
where product_id is not null
```

silver_subsidiary_master.sql

silver_subsidiary_master.sql

```
{{ config(materialized='view') }}

select
    subsidiary_id,
    initcap(trim(subsidiary_name)) as subsidiary_name,
    upper(trim(functional_currency)) as functional_currency
from {{ ref('bronze_subsidiary_master') }}
where subsidiary_id is not null
```

silver_chart_of_accounts.sql

silver_chart_of_accounts.sql

```
{{ config(materialized='view') }}

select
    account_number,
    initcap(trim(account_name)) as account_name,
    upper(trim(account_type)) as account_type,
    upper(trim(sub_type)) as sub_type,
    upper(trim(normal_balance)) as normal_balance,
    upper(trim(financial_statement_category)) as financial_statement_category,
    case
        when upper(trim(financial_statement_category)) = 'BALANCE SHEET' then 'BS'
        when upper(trim(financial_statement_category)) = 'INCOME STATEMENT' then 'IS'
        else 'OTHER'
    end as statement_code
from {{ ref('bronze_chart_of_accounts') }}
where account_number is not null
```

silver_well_production.sql

silver_well_production.sql

```
{{ config(materialized='view') }}

select
    well_id,
    well_name,
    production_date,
    oil_volume_bbl,
    gas_volume_mcf,
    water_volume_bbl,
    oil_volume_bbl + water_volume_bbl as total_liquid_bbl,
    case
        when (oil_volume_bbl + water_volume_bbl) > 0
        then water_volume_bbl / (oil_volume_bbl + water_volume_bbl) * 100
        else 0
    end as water_cut_pct,
    wellhead_pressure_psi,
    wellhead_temperature_f,
    choke_size,
    status
from {{ ref('bronze_well_production') }}
where status is not null
```

silver_pipeline_operations.sql

silver_pipeline_operations.sql

```
{{ config(materialized='view') }}

select
    pipeline_id,
    pipeline_name,
    segment_id,
    reading_timestamp,
    inlet_pressure_psi,
    outlet_pressure_psi,
    inlet_pressure_psi - outlet_pressure_psi as pressure_drop_psi,
    flow_rate_bpd,
    temperature_f,
    status,
    case
        when inlet_pressure_psi - outlet_pressure_psi > 50 then 'HIGH'
        when inlet_pressure_psi - outlet_pressure_psi > 20 then 'MEDIUM'
        else 'NORMAL'
    end as pressure_drop_severity
from {{ ref('bronze_pipeline_operations') }}
where status is not null
```

6. Gold Layer - Star Schema

The Gold layer implements a star schema with Fact and Dimension tables, materialized as tables for optimal query performance.

Schema & Tests

_gold_schema.yml

```
version: 2

models:
  - name: gold_daily_production_summary
    description: Daily aggregated production metrics
    columns:
      - name: production_date
        tests:
          - unique
          - not_null
      - name: active_wells
        tests:
          - not_null
      - name: total_oil_bbl
        tests:
          - not_null

  - name: gold_pipeline_health
    description: Pipeline-level health and severity summary
    columns:
      - name: pipeline_id
        tests:
          - unique
          - not_null
      - name: pipeline_name
        tests:
          - not_null
```

fact_general_ledger.sql

fact_general_ledger.sql

```
{{ config(materialized='table') }}

SELECT
    gl.journal_id,
    gl.line_id,
    gl.posting_date,
    gl.document_date,
    gl.subsidiary_id,
    gl.account_number,
    gl.customer_id,
    gl.vendor_id,
    gl.product_id,
    gl.quantity,
    COALESCE(gl.debit_amount, 0) AS debit_amount,
    COALESCE(gl.credit_amount, 0) AS credit_amount,
    COALESCE(gl.debit_amount, 0) - COALESCE(gl.credit_amount, 0) AS net_amount,
    gl.created_by,
    gl.approval_status,
    gl.error_flag,
    gl.error_type
FROM {{ ref('bronze_gl') }} gl
```

fact_vendor_payments.sql

fact_vendor_payments.sql

```
{{ config(materialized='table') }}

SELECT
    gl.journal_id,
    gl.posting_date,
    gl.vendor_id,
    gl.subsidiary_id,
    gl.account_number,
    COALESCE(gl.debit_amount, 0) AS payment_amount,
    gl.quantity,
    gl.created_by,
    gl.approval_status
FROM {{ ref('bronze_gl') }} gl
WHERE gl.vendor_id IS NOT NULL
    AND COALESCE(gl.debit_amount, 0) > 0
```

fact_revenue.sql

fact_revenue.sql

```
{{ config(materialized='table') }}

SELECT
    gl.journal_id,
    gl.posting_date,
    gl.customer_id,
    gl.product_id,
    gl.subsidiary_id,
    gl.account_number,
    COALESCE(gl.credit_amount, 0) AS revenue_amount,
    gl.quantity,
    CASE
        WHEN gl.quantity > 0 AND pm.standard_cost_per_barrel IS NOT NULL
        THEN gl.quantity * pm.standard_cost_per_barrel
        ELSE 0
    END AS standard_cost,
    COALESCE(gl.credit_amount, 0) - CASE
        WHEN gl.quantity > 0 AND pm.standard_cost_per_barrel IS NOT NULL
        THEN gl.quantity * pm.standard_cost_per_barrel
        ELSE 0
    END AS gross_margin,
    gl.created_by
FROM {{ ref('bronze_gl') }} gl
LEFT JOIN {{ ref('bronze_product_master') }} pm
    ON gl.product_id = pm.product_id
WHERE gl.customer_id IS NOT NULL
    AND COALESCE(gl.credit_amount, 0) > 0
```

gold_daily_production_summary.sql

gold_daily_production_summary.sql

```
{{ config(materialized='table') }}

select
    production_date,
    count(distinct well_id) as active_wells,
    sum(oil_volume_bbl) as total_oil_bbl,
    sum(gas_volume_mcf) as total_gas_mcf,
    sum(water_volume_bbl) as total_water_bbl,
    sum(total_liquid_bbl) as total_liquid_bbl,
    avg(water_cut_pct) as avg_water_cut_pct,
    avg(wellhead_pressure_psi) as avg_wellhead_pressure_psi
from {{ ref('silver_well_production') }}
group by production_date
```

gold_pipeline_health.sql

gold_pipeline_health.sql

```
{{ config(materialized='table') }}

select
    pipeline_id,
    pipeline_name,
    count(distinct segment_id) as total_segments,
    avg(pressure_drop_psi) as avg_pressure_drop_psi,
    max(pressure_drop_psi) as max_pressure_drop_psi,
    avg(flow_rate_bpd) as avg_flow_rate_bpd,
    sum(case when pressure_drop_severity = 'HIGH' then 1 else 0 end) as high_severity_count,
    sum(case when pressure_drop_severity = 'MEDIUM' then 1 else 0 end) as medium_severity_count
from {{ ref('silver_pipeline_operations') }}
group by pipeline_id, pipeline_name
```

dim_vendor.sql

dim_vendor.sql

```
{{ config(materialized='table') }}

SELECT
    vendor_id,
    vendor_name,
    service_type,
    location
FROM {{ ref('bronze_vendor_master') }}
```

dim_customer.sql

dim_customer.sql

```
{{ config(materialized='table') }}

SELECT
    customer_id,
    customer_name,
    credit_limit,
    industry,
    location
FROM {{ ref('bronze_customer_master') }}
```

dim_product.sql

dim_product.sql

```
{{ config(materialized='table') }}

SELECT
    product_id,
    product_name,
    category,
    standard_cost_per_barrel
FROM {{ ref('bronze_product_master') }}
```

dim_subsidiary.sql

dim_subsidiary.sql

```
{{ config(materialized='table') }}

SELECT
    subsidiary_id,
    subsidiary_name,
    functional_currency
FROM {{ ref('bronze_subsidiary_master') }}
```

dim_chart_of_accounts.sql

dim_chart_of_accounts.sql

```
{{ config(materialized='table') }}

SELECT
    account_number,
    account_name,
    account_type,
    sub_type,
    normal_balance,
    financial_statement_category
FROM {{ ref('bronze_chart_of_accounts') }}
```

dim_date.sql

dim_date.sql

```
{{ config(materialized='table') }}

WITH date_spine AS (
    SELECT
        DATEADD(day, seq4(), '2020-01-01')::DATE AS date_key
    FROM TABLE(GENERATOR(ROWCOUNT => 3650))
)

SELECT
    date_key,
    YEAR(date_key) AS year,
    QUARTER(date_key) AS quarter,
    MONTH(date_key) AS month,
    DAY(date_key) AS day,
    DAYOFWEEK(date_key) AS day_of_week,
    DAYNAME(date_key) AS day_name,
    MONTHNAME(date_key) AS month_name,
    CONCAT('Q', QUARTER(date_key)) AS quarter_name,
    CONCAT(YEAR(date_key), '-Q', QUARTER(date_key)) AS year_quarter,
    CASE WHEN DAYOFWEEK(date_key) IN (0, 6) THEN FALSE ELSE TRUE END AS is_weekday
FROM date_spine
```

7. Snapshots (SCD Type 2)

The vendor_snapshot captures slowly changing dimensions using the 'check' strategy. It monitors vendor_name, service_type, and location for changes and maintains history with dbt_valid_from and dbt_valid_to columns.

vendor_snapshot.sql

```
{% snapshot vendor_snapshot %}

{{
    config(
        target_schema='snapshots',
        unique_key='vendor_id',
        strategy='check',
        check_cols=['vendor_name', 'service_type', 'location']
    )
}}

SELECT *
FROM {{ source('bronze','vendor_master') }}

{% endsnapshot %}
```

8. Tests & Data Quality

28 schema tests + 4 custom singular tests ensure data quality across all layers.

Schema Tests Summary

Test Type	Count	Applied To
not_null	18	All primary keys & critical columns
unique	2	Gold layer primary keys
accepted_values	3	Status & severity fields
relationships	1	GL account_number -> COA

Custom Tests

trial_balance.sql - Debits must equal credits per journal entry

trial_balance.sql

```
SELECT journal_entry_id

FROM {{ ref('gl_transactions_enriched') }}

GROUP BY journal_entry_id

HAVING SUM(debit_amount) != SUM(credit_amount)
```

assert_no_negative_oil_volume.sql - No negative oil production

assert_no_negative_oil_volume.sql

```
-- Wells should never report negative oil volumes
select
    well_id,
    production_date,
    oil_volume_bbl
from {{ ref('silver_well_production') }}
where oil_volume_bbl < 0
```

assert_no_negative_pressure_drop.sql - Outlet pressure <= inlet pressure

assert_no_negative_pressure_drop.sql

```
-- Outlet pressure should not exceed inlet pressure (negative pressure drop)
select
    pipeline_id,
    segment_id,
    reading_timestamp,
    pressure_drop_psi
from {{ ref('silver_pipeline_operations') }}
where pressure_drop_psi < 0
```

assert_water_cut_in_range.sql - Water cut 0-100%

assert_water_cut_in_range.sql

```
-- Water cut percentage should be between 0 and 100
select
    well_id,
    production_date,
    water_cut_pct
from {{ ref('silver_well_production') }}
where water_cut_pct < 0 or water_cut_pct > 100
```

9. Seeds - Sample Data

seed_properties.yml

```
version: 2

seeds:
  - name: seed_well_production
    description: Sample well production data for development and testing
  - name: seed_pipeline_operations
    description: Sample pipeline operations data for development and testing
```

seed_well_production.csv

```
well_id,well_name,production_date,oil_volume_bbl,gas_volume_mcf,water_volume_bbl,wellhead_pressure_psi,wellhead_temperature_f,
W001,Alpha Well,2024-01-01,150.5,320.0,45.2,2800,185,32,ACTIVE,2024-01-02 08:00:00
W001,Alpha Well,2024-01-02,148.3,315.5,46.8,2785,183,32,ACTIVE,2024-01-03 08:00:00
W001,Alpha Well,2024-01-03,152.1,325.0,44.5,2810,186,32,ACTIVE,2024-01-04 08:00:00
W002,Beta Well,2024-01-01,210.0,450.2,78.3,3100,192,48,ACTIVE,2024-01-02 08:00:00
W002,Beta Well,2024-01-02,205.8,442.0,80.1,3085,190,48,ACTIVE,2024-01-03 08:00:00
W002,Beta Well,2024-01-03,208.5,448.5,79.0,3095,191,48,ACTIVE,2024-01-04 08:00:00
W003,Gamma Well,2024-01-01,95.2,180.0,120.5,2200,175,24,ACTIVE,2024-01-02 08:00:00
W003,Gamma Well,2024-01-02,92.0,175.5,125.0,2180,174,24,ACTIVE,2024-01-03 08:00:00
W003,Gamma Well,2024-01-03,0.0,0.0,0.0,0,0,0,SHUT_IN,2024-01-04 08:00:00
W004,Delta Well,2024-01-01,180.0,390.0,55.0,2950,188,40,ACTIVE,2024-01-02 08:00:00
W004,Delta Well,2024-01-02,175.5,385.0,58.2,2930,187,40,ACTIVE,2024-01-03 08:00:00
W004,Delta Well,2024-01-03,178.0,388.0,56.5,2940,188,40,ACTIVE,2024-01-04 08:00:00
```

seed_pipeline_operations.csv

```
pipeline_id,pipeline_name,segment_id,reading_timestamp,inlet_pressure_psi,outlet_pressure_psi,flow_rate_bpd,temperature_f,stat
P001,North Trunk Line,SEG-001,2024-01-01 06:00:00,850,820,5200,140,OPERATIONAL,2024-01-01 08:00:00
P001,North Trunk Line,SEG-002,2024-01-01 06:00:00,820,780,5200,138,OPERATIONAL,2024-01-01 08:00:00
P001,North Trunk Line,SEG-003,2024-01-01 06:00:00,780,710,5150,135,OPERATIONAL,2024-01-01 08:00:00
P001,North Trunk Line,SEG-001,2024-01-02 06:00:00,855,825,5250,141,OPERATIONAL,2024-01-02 08:00:00
P001,North Trunk Line,SEG-002,2024-01-02 06:00:00,825,785,5250,139,OPERATIONAL,2024-01-02 08:00:00
P001,North Trunk Line,SEG-003,2024-01-02 06:00:00,785,715,5200,136,OPERATIONAL,2024-01-02 08:00:00
P002,South Feeder,SEG-001,2024-01-01 06:00:00,920,850,3800,145,OPERATIONAL,2024-01-01 08:00:00
P002,South Feeder,SEG-002,2024-01-01 06:00:00,850,840,3800,143,OPERATIONAL,2024-01-01 08:00:00
P002,South Feeder,SEG-001,2024-01-02 06:00:00,925,855,3850,146,OPERATIONAL,2024-01-02 08:00:00
P002,South Feeder,SEG-002,2024-01-02 06:00:00,855,845,3850,144,OPERATIONAL,2024-01-02 08:00:00
P003,East Connector,SEG-001,2024-01-01 06:00:00,800,740,2900,132,OPERATIONAL,2024-01-01 08:00:00
P003,East Connector,SEG-001,2024-01-02 06:00:00,810,745,2950,133,OPERATIONAL,2024-01-02 08:00:00
```

10. Macros

Custom generate_schema_name macro overrides dbt's default behavior of prefixing schemas with target_schema. Models deploy to exact schema names: bronze, silver, gold.

generate_schema_name.sql

```
{% macro generate_schema_name(custom_schema_name, node) -%}

    {%- set default_schema = target.schema -%}

    {%- if custom_schema_name is none -%}
        {{ default_schema }}
    {%- else -%}
        {{ custom_schema_name | trim }}
    {%- endif -%}

{%- endmacro %}
```

11. CI/CD - GitHub Actions

The pipeline is automated via GitHub Actions with daily scheduled runs at 6 AM UTC and manual trigger support with optional full-refresh. Each layer runs and tests sequentially: Seed -> Bronze -> Silver -> Gold.

dbt_pipeline.yml

```
name: dbt Petroleum Pipeline

on:
  # Scheduled run - Daily at 6 AM UTC
  schedule:
    - cron: '0 6 * * *'

  # Manual trigger
  workflow_dispatch:
    inputs:
      full_refresh:
        description: 'Run with --full-refresh'
        required: false
        default: 'false'
        type: choice
        options:
          - 'false'
          - 'true'

env:
  SNOWFLAKE_PASSWORD: ${{ secrets.SNOWFLAKE_PASSWORD }}

defaults:
  run:
    working-directory: petroleum_pipeline

jobs:
  dbt-seed:
    name: dbt Seed
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.12'

      - name: Install dbt
        run: pip install dbt-snowflake==1.11.3
        working-directory: .

      - name: dbt deps
        run: dbt deps --profiles-dir .

      - name: dbt seed
        run: dbt seed --profiles-dir .

  dbt-bronze:
    name: Bronze Layer
    needs: dbt-seed
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
```

```
    - name: Set up Python
      uses: actions/setup-python@v5
      with:
        python-version: '3.12'

    - name: Install dbt
      run: pip install dbt-snowflake==1.11.3
      working-directory: .

    - name: dbt snapshot
      run: dbt snapshot --profiles-dir .

    - name: dbt run bronze
      run: |
        if [ "${{ github.event.inputs.full_refresh }}" = "true" ]; then
          dbt run --select models/bronze --full-refresh --profiles-dir .
        else
          dbt run --select models/bronze --profiles-dir .
        fi

    - name: dbt test bronze
      run: dbt test --select models/bronze --profiles-dir .

  dbt-silver:
    name: Silver Layer
    needs: dbt-bronze
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.12'

      - name: Install dbt
        run: pip install dbt-snowflake==1.11.3
        working-directory: .

      - name: dbt run silver
        run: dbt run --select models/silver --profiles-dir .

      - name: dbt test silver
        run: dbt test --select models/silver --profiles-dir .

  dbt-gold:
    name: Gold Layer
    needs: dbt-silver
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.12'

      - name: Install dbt
        run: pip install dbt-snowflake==1.11.3
        working-directory: .

      - name: dbt run gold
        run: dbt run --select models/gold --profiles-dir .
```

```
      - name: dbt test gold
        run: dbt test --select models/gold --profiles-dir .

      - name: dbt docs generate
        run: dbt docs generate --profiles-dir .
```

12. Orchestration - Apache Airflow

An Airflow DAG orchestrates the dbt pipeline with 10 tasks running sequentially. Docker Compose is provided for local Airflow deployment.

Airflow DAG

petroleum_pipeline_dag.py

```
from datetime import datetime, timedelta
from airflow import DAG
from airflow.operators.bash import BashOperator

# Path to your dbt project
DBT_PROJECT_DIR = "c:/dev_work/petroleum_pipeline/petroleum_pipeline"

default_args = {
    "owner": "rahul",
    "depends_on_past": False,
    "retries": 2,
    "retry_delay": timedelta(minutes=5),
}

with DAG(
    dag_id="petroleum_pipeline",
    default_args=default_args,
    description="dbt Petroleum Pipeline - Bronze to Silver to Gold",
    schedule="0 6 * * *",  # Daily at 6 AM UTC
    start_date=datetime(2024, 1, 1),
    catchup=False,
    tags=["dbt", "petroleum", "snowflake"],
) as dag:

    env = {"SNOWFLAKE_PASSWORD": "{{ var.value.snowflake_password }}"}

    # Step 1: dbt deps - install packages
    dbt_deps = BashOperator(
        task_id="dbt_deps",
        bash_command=f"cd {DBT_PROJECT_DIR} && dbt deps",
        env=env,
    )

    # Step 2: dbt seed - load seed data
    dbt_seed = BashOperator(
        task_id="dbt_seed",
        bash_command=f"cd {DBT_PROJECT_DIR} && dbt seed",
        env=env,
    )

    # Step 3: dbt snapshot - SCD Type 2
    dbt_snapshot = BashOperator(
        task_id="dbt_snapshot",
        bash_command=f"cd {DBT_PROJECT_DIR} && dbt snapshot",
        env=env,
    )

    # Step 4: Bronze layer - incremental loads
    dbt_run_bronze = BashOperator(
        task_id="dbt_run_bronze",
        bash_command=f"cd {DBT_PROJECT_DIR} && dbt run --select models/bronze",
        env=env,
```

```
)

# Step 5: Bronze tests
dbt_test_bronze = BashOperator(
    task_id="dbt_test_bronze",
    bash_command=f"cd {DBT_PROJECT_DIR} && dbt test --select models/bronze",
    env=env,
)

# Step 6: Silver layer
dbt_run_silver = BashOperator(
    task_id="dbt_run_silver",
    bash_command=f"cd {DBT_PROJECT_DIR} && dbt run --select models/silver",
    env=env,
)

# Step 7: Silver tests
dbt_test_silver = BashOperator(
    task_id="dbt_test_silver",
    bash_command=f"cd {DBT_PROJECT_DIR} && dbt test --select models/silver",
    env=env,
)

# Step 8: Gold layer - star schema
dbt_run_gold = BashOperator(
    task_id="dbt_run_gold",
    bash_command=f"cd {DBT_PROJECT_DIR} && dbt run --select models/gold",
    env=env,
)

# Step 9: Gold tests
dbt_test_gold = BashOperator(
    task_id="dbt_test_gold",
    bash_command=f"cd {DBT_PROJECT_DIR} && dbt test --select models/gold",
    env=env,
)

# Step 10: Generate dbt docs
dbt_docs_generate = BashOperator(
    task_id="dbt_docs_generate",
    bash_command=f"cd {DBT_PROJECT_DIR} && dbt docs generate",
    env=env,
)

# DAG flow
dbt_deps >> dbt_seed >> dbt_snapshot >> dbt_run_bronze >> dbt_test_bronze
dbt_test_bronze >> dbt_run_silver >> dbt_test_silver
dbt_test_silver >> dbt_run_gold >> dbt_test_gold >> dbt_docs_generate
```

Docker Compose

docker-compose.yml

```
version: '3.8'

x-airflow-common: &airflow-common
  image: apache/airflow:2.8.1-python3.10
  environment:
    AIRFLOW__CORE__EXECUTOR: LocalExecutor
    AIRFLOW__DATABASE__SQL_ALCHEMY_CONN: postgresql+psycopg2://airflow:airflow@postgres/airflow
    AIRFLOW__CORE__FERNET_KEY: ''
    AIRFLOW__CORE__DAGS_ARE_PAUSED_AT_CREATION: 'true'
    AIRFLOW__CORE__LOAD_EXAMPLES: 'false'
    SNOWFLAKE_PASSWORD: ${SNOWFLAKE_PASSWORD}
  volumes:
    - ./dags:/opt/airflow/dags
    - ./logs:/opt/airflow/logs
    - ../petroleum_pipeline:/opt/airflow/dbt/petroleum_pipeline
  depends_on:
    postgres:
      condition: service_healthy

services:
  postgres:
    image: postgres:15
    environment:
      POSTGRES_USER: airflow
      POSTGRES_PASSWORD: airflow
      POSTGRES_DB: airflow
    healthcheck:
      test: ["CMD", "pg_isready", "-U", "airflow"]
      interval: 10s
      retries: 5
    ports:
      - "5432:5432"

  airflow-init:
    <<: *airflow-common
    entrypoint: /bin/bash
    command:
      - -c
      - |
        pip install dbt-snowflake==1.11.3 &&
        airflow db migrate &&
        airflow users create \
          --username admin \
          --password admin \
          --firstname Rahul \
          --lastname Tiwary \
          --role Admin \
          --email rahul@example.com
    restart: "no"

  airflow-webserver:
    <<: *airflow-common
    command: bash -c "pip install dbt-snowflake==1.11.3 && airflow webserver"
    ports:
      - "8080:8080"
    restart: always

  airflow-scheduler:
    <<: *airflow-common
```

```
    command: bash -c "pip install dbt-snowflake==1.11.3 && airflow scheduler"
    restart: always

volumes:
  postgres-db-volume:
```

13. End-to-End Workflow

Step 1: Environment Setup

```
# Install dbt
pip install dbt-snowflake

# Set Snowflake password
export SNOWFLAKE_PASSWORD='<your_password>'

# Verify connection
cd petroleum_pipeline
dbt debug
```

Step 2: Load Seed Data

```
dbt seed
# Loads seed_well_production (12 rows)
# Loads seed_pipeline_operations (12 rows)
```

Step 3: Run Snapshots (SCD Type 2)

```
dbt snapshot
# Captures vendor_master state for change tracking
```

Step 4: Build Bronze Layer (Incremental)

```
dbt run --select models/bronze
# Incremental loads from PETROLEUM_AUDIT.BRONZE
# 6 incremental models + 3 views

dbt test --select models/bronze
# Tests: not_null, accepted_values on status fields
```

Step 5: Build Silver Layer (Transform)

```
dbt run --select models/silver
# Joins, cleaning, standardization, business logic
# gl_transactions_enriched, vendor_payments, etc.

dbt test --select models/silver
# Tests: not_null, relationships, trial_balance
```

Step 6: Build Gold Layer (Star Schema)

```
dbt run --select models/gold
# Fact tables: fact_general_ledger, fact_revenue, etc.
# Dimension tables: dim_vendor, dim_customer, dim_date, etc.

dbt test --select models/gold
# Tests: unique, not_null on primary keys
```

Step 7: Full Build (All at Once)

```
dbt build
```

```
# Runs: seeds + models + tests + snapshots
# Result: 61/61 PASS
```

Step 8: Load New Data from S3

```
# New data arrives in s3://petroleumdb/vendors.csv
# Snowflake stage: petroleum_stage

COPY INTO vendor_master
FROM @petroleum_stage/vendors.csv
FILE_FORMAT = (TYPE='CSV' SKIP_HEADER=1);

# Run incremental model + downstream
dbt run --select bronze_vendor_master+
```

Step 9: CI/CD Automation

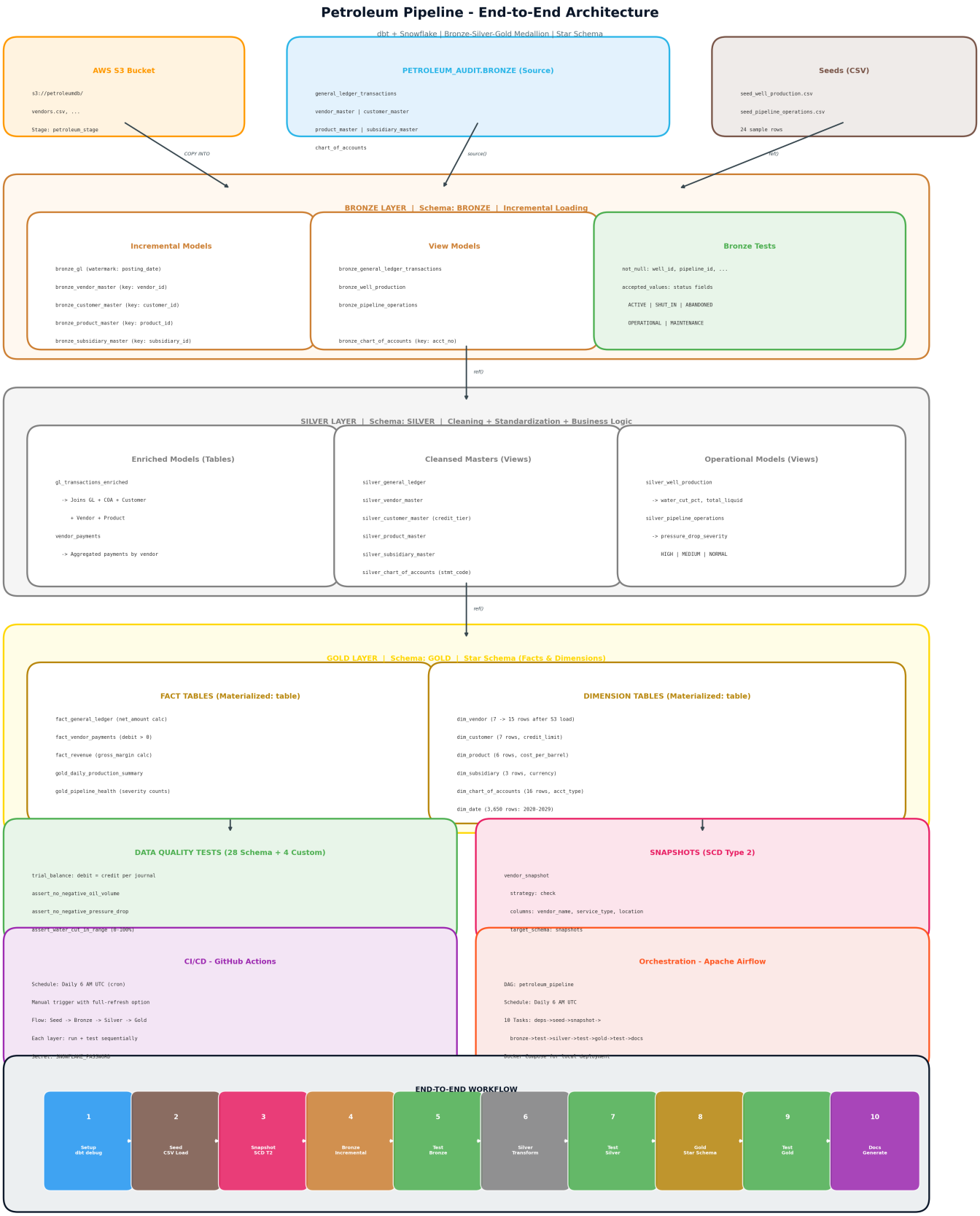
```
# GitHub Actions runs daily at 6 AM UTC
# Manual trigger: GitHub Actions > Run workflow
# Secret required: SNOWFLAKE_PASSWORD in repo settings
```

14. Commands Reference

Command	Description
dbt debug	Test Snowflake connection
dbt seed	Load CSV seed data
dbt run	Run all models
dbt test	Run all tests
dbt snapshot	Run SCD Type 2 snapshots
dbt build	Seed + Run + Test + Snapshot
dbt run --select model_name	Run specific model
dbt run --select model_name+	Model + all downstream
dbt run --full-refresh	Rebuild incremental models
dbt docs generate	Generate documentation
dbt docs serve	Serve docs locally

15. Architecture Diagram

Complete end-to-end architecture of the Petroleum Pipeline project:



Repository: github.com/tiwaryrahul/Petroleum- | Database: Snowflake | Tool: dbt 1.11.7

Result: 61/61 PASS | 30 Models | 28 Tests | 2 Seeds | 1 Snapshot | 6 Sources